Inactive & Legacy Data Cleanup & Storage Optimization — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 4

The problem . . . 4

The solution . . . 4

What it delivers . . . 4

Business outcomes . . . 5

Boundaries (so scope is clear) . . . 5

1. Solution Overview — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 6

How it works . . . 7

The in-scope workloads (and what cleanup means for each) . . . 7

The identify→clean-up-defensibly journey (the core discipline) . . . 8

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 9

The standard deployment model . . . 9

2. Requirements — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 9

2.1 Licensing sufficient for the lifecycle features you will use (E3 baseline vs E5) . . . 10

2.2 Data-lifecycle / records least-privilege roles . . . 10

2.3 A storage / ROT baseline and the signals to compute it . . . 11

2.4 An inventory of existing holds (litigation, eDiscovery case, retention) — the safety gate . . . 11

2.5 A records / retention-schedule decision source (deferred, but required as input) . . . 12

2.6 A limited pilot scope, adaptive-scope attribute data, and change control . . . 12

2.7 A disposition review board and workflow (where deletion needs a human gate) . . . 13

Global Requirements — Microsoft Purview Solutions 14

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 14

2. Purview roles (least-privilege) and portal access . . . 14

3. Security & Compliance PowerShell + Graph toolchain . . . 15

4. Data sources & scope in scope . . . 15

5. Compliance, records, privacy & legal stakeholders . . . 15

6. A pilot scope and change control . . . 15

3. Implementation Guide — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 16

Phase 0 — Connect and baseline the storage / ROT posture (Assess) . . . 16
Phase 1 — Design the labels, policies, and scope (Design — with the records function) . . . 17
Phase 2 — Configure and publish/auto-apply in SIMULATION (Configure — no deletion) . . . 18
Phase 3 — Validate scope and prove no held item is deletable (Validate — the safety gate) . . . 18
Phase 4 — Pilot the cleanup with `-WhatIf` and disposition review (Pilot) . . . 19
Phase 5 — Expand under change control and operate (Expand → Operate) . . . 20
Order-of-operations summary . . . 20
5. Testing & Validation — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 21
Test A — Identification & scoping (the right data is targeted) . . . 21
Test B — Hold safety (no held item is deletable) — the defensibility gate . . . 22
Test C — Controlled cleanup & disposition evidence (acts only where intended) . . . 23
Test D — Savings measurement & reversibility . . . 23
Evidence to capture (for the deliverable record) . . . 24
6. Troubleshooting — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 24
Nothing seems to happen . . . 25
Wrong content targeted / too much in scope . . . 25
Held content is unexpectedly in scope (the safety gate) . . . 26
Labels / policies won't create or apply . . . 26
Reports / discovery come back empty . . . 26
Something changed unexpectedly . . . 27
7. Best Practices — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 28
Check holds first, every time — the defensibility gate . . . 28
Never auto-delete — pilot small, preview with `-WhatIf`, expand slowly . . . 28
Use simulation and disposition review as human gates . . . 29
Understand retention precedence before you design . . . 29
Get the scope right — static for pilot, adaptive for scale . . . 29
Measure ROT and savings from evidence . . . 29
Mind the platform's behavior . . . 30
Stay inside the boundary . . . 30
8. Deliverables Summary — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview) 30
Documentation . . . 30
PowerShell tools (5) — Security & Compliance PowerShell / Exchange Online / Graph . . . 31
Definition of done . . . 33
Appendix A — PowerShell Scripts 34

Executive Summary — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

The problem

Every tenant quietly accumulates **ROT** — data that is **Redundant, Obsolete, or Trivial** — and **inactive / legacy** content: the mailbox of an employee who left years ago, a SharePoint site nobody has opened since a finished project, an unused OneDrive, a Teams team for an event long past, a decade of duplicate "final_v7" files. It inflates **storage cost**, enlarges the **data-breach blast radius** (every stale file is one more thing to exfiltrate), and drags on **eDiscovery and audits**. The temptation is to "just delete the old stuff" — but deletion is **irreversible**, and deleting content that turns out to be **under legal hold** or subject to a **retention obligation** is a serious, sometimes legal, failure.

The solution

Use **Microsoft Purview Data Lifecycle Management and Records Management** — the native retention/deletion engine — to **identify** ROT and stale/legacy data and **defensibly clean it up** across **Exchange, SharePoint, OneDrive, Teams, and Microsoft 365 Groups**, using only the Microsoft Purview portal, Security & Compliance PowerShell, Exchange Online PowerShell, and Microsoft Graph usage reports. The defining discipline is that **nothing is ever auto-deleted and everything is proven safe first**: stale-data candidates are **identified and sized from real signals**; auto-apply label policies run in **simulation**; deletion policies are **piloted on a limited scope** and previewed with `-WhatIf`; **every existing hold is enumerated and cross-checked** so no held item can be deleted; and record-value content passes through **disposition review** that produces an auditable approval trail. The result: storage and exposure go **down**, defensibly, without destroying anything that should have been kept.

What it delivers

- A **sized ROT candidate inventory** — inactive mailboxes, dormant sites, unused OneDrives, orphaned

Teams/Groups — built from mailbox/site last-activity and storage signals, not guesswork.

- **Retention labels and retention/deletion policies** (retain / retain-then-delete / delete-only /

disposition-review) scoped by **static or adaptive** scope, designed with the records function.

- A **hold safety gate**: the complete hold inventory (litigation, eDiscovery case, retention) cross-checked

against the cleanup scope, proving **no held item is deletable** before enforcement.

- **Disposition review** for record-value content, producing the **who-approved-deleting-what-and-when**

evidence that makes cleanup defensible.

- **Five PowerShell tools** (posture/hold discovery, policy config helper, validation/hold cross-check,

ROT/savings report, scope helper) — read-only or `-WhatIf` by default; **none deletes content**.

Business outcomes

• **Lower storage cost and a smaller attack surface** — ROT and inactive data reduced across the workloads it

actually lives in.

• **Defensible, not reckless** — the hold cross-check and disposition review mean cleanup can be evidenced to

auditors and counsel, not just performed.

• **No accidental data loss** — limited-scope pilots, `-WhatIf` previews, simulation, and hold checks make the

control reversible right up to the point of authorized permanent deletion.

• **Self-guided at a fraction of consulting cost** — the same method a specialist would use, documented and

tooled to run in-house.

Boundaries (so scope is clear)

Feature availability is licensing-/plan-dependent — **adaptive scopes, auto-apply, Records Management, and disposition review generally require Microsoft 365 E5 / E5 Compliance** (some preview); basic labels and static retention are E3-capable (verify your tenant). This solution provides the **technical method and the evidence**; *how long* to keep each class, *what* is under legal hold, whether content is a **record**, and whether a given deletion is **authorized and defensible** are **records, compliance, privacy, and legal determinations**, captured alongside the delivered evidence.

1. Solution Overview — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

What this solution is

Every tenant accumulates **ROT** — data that is **Redundant, Obsolete, or Trivial**: the mailbox of an employee who left three years ago, a SharePoint site nobody has opened since a finished project, a decade of duplicate drafts and "final_v7" attachments, a Teams team for an event that already happened. Left alone, ROT and **inactive/legacy data** quietly inflate **storage cost**, enlarge the **attack surface and data-breach blast radius**, and complicate **eDiscovery and audits** — every stale file is one more thing an attacker can exfiltrate and one more thing counsel has to search. **Microsoft Purview Data Lifecycle Management (DLM)** and **Records Management** are the native way to fix this: **retention labels** and **retention/deletion policies** that keep what must be kept and **defensibly delete** what may go, across **Exchange, SharePoint, OneDrive, Teams, and Microsoft 365 Groups** — driven by **static or adaptive scopes**, with **disposition review** and a full **audit trail** for anything sensitive.

This is the **inactive & legacy data cleanup and storage-optimization solution**: a repeatable, evidence-driven way to **identify** ROT and stale/legacy data (inactive mailboxes, dormant sites, unused OneDrives, orphaned Teams/Groups, existing retention posture and holds), **design** the retention/deletion controls that clean it up, and **execute** the cleanup **defensibly** — using only the **Microsoft Purview portal**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), **Exchange Online PowerShell**, and

Microsoft Graph (usage reports). Its defining discipline is that **deletion is high-impact and irreversible, so nothing is auto-deleted and everything is proven safe first** — auto-apply label policies run in **simulation**, deletion policies are piloted on a **limited scope**, scripts run **read-only or** `-WhatIf`, and **every existing hold is checked and respected** before a single item is removed.

> ***Scope boundary — retention-schedule and legal determinations are deferred.*** *This solution delivers the **technical method and the evidence** for identifying and cleaning up ROT/inactive/legacy data. *How long* each data class must be kept, *which* records are subject to a legal or regulatory retention obligation, *what* is under **legal hold**, and *whether* a given deletion is defensible are **records-management, compliance, privacy, and legal determinations** — deferred to those functions. The solution finds the candidates, configures the control, proves nothing held would be deleted, and produces the disposition evidence; it does **not** decide the retention schedule or authorize the deletion.*

Why this matters

• **Deletion is irreversible and high-impact.** A retention policy set to "delete after N days" over the wrong

scope can purge live business content permanently. This is the opposite of DLP (which blocks) — a cleanup mistake destroys data. That is why every control here is **piloted on a limited scope, run in** `-WhatIf`**, and hold-checked** before enforcement, and why **nothing is ever auto-deleted** by these tools.

• **Holds must win, and you must prove it.** Content under a **litigation hold**, an **eDiscovery case hold**,

or a retain-action retention policy is **protected from deletion** by Purview's retention principles — a delete policy will *not* purge held items. But "the platform should protect it" is not evidence; this solution **enumerates the holds and validates** that no in-scope item under hold would be removed **before** you enforce.

• **ROT is invisible until you measure it.** Inactive mailboxes, dormant sites, and unused OneDrives don't

announce themselves. Storage and activity **reporting** (mailbox/site last-activity, storage consumed, inactive-mailbox inventory) is what turns "we probably have a lot of junk" into a named, sized candidate list the records function can act on.

• **Retention keeps what deletion would otherwise remove.** The same lifecycle engine that deletes ROT also

retains what must be kept — the two are one system governed by the **principles of retention** (retention wins over deletion; the longest retention wins; explicit beats implicit; a hold blocks purge). Designing cleanup without understanding retention precedence is how you either delete too much or delete nothing.

• **Adaptive scopes keep cleanup targeted as the estate changes.** A static list of stale sites is stale the

day after you write it. **Adaptive scopes** (query-based membership over user/site/group attributes) let a policy automatically target, say, sites with no recent activity or mailboxes of departed users — without hand-maintaining membership.

How it works

1. **Discovery / storage reporting** identifies *what* is stale and *how big* it is: existing **retention

policies, retention labels, and label policies**;** holds **in force (litigation, eDiscovery case, retention);** inactive mailboxes **(**`Get-Mailbox -InactiveMailboxOnly`**); and mailbox/site/OneDrive** last-activity and storage **signals (Exchange mailbox statistics and** Microsoft Graph usage reports**). This is the ROT candidate inventory.

1. **Retention labels** define *what happens to a class of content* — **retain**, **retain then delete**, or

delete-only — and can require **disposition review** (a human approval step) before permanent deletion. Labels are the precise, item-level instrument (they can carry **records** declaration where the records function requires immutability).

1. **Retention (and deletion) policies** apply a retain/delete action **in bulk** across a **location** (Exchange

mailboxes, SharePoint sites, OneDrive accounts, Teams chats/channels, M365 Groups) using a **static scope** (named locations) or an **adaptive scope** (attribute query).

1. **Auto-apply label policies** stamp a retention label onto content that matches a condition (sensitive info

type, keyword/query, or trainable classifier) — and **support a simulation mode** so you can see *what would be labeled* before anything is applied.

1. **Piloting** proves the control on a **limited scope** first — a handful of pilot sites/mailboxes, auto-apply

in **simulation**, deletion actions previewed with the delivered `-WhatIf` tooling — with a **hold-safety check** that no in-scope item is under hold.

1. **Disposition review** routes items reaching end-of-life to **reviewers** who approve, extend, or relabel —

producing the **disposition evidence** (who approved deleting what, and when) that makes cleanup defensible.

1. **Reporting** produces the **storage-savings / candidate-ROT / disposition** report for the records and

storage owners to review and sign off.

The in-scope workloads (and what cleanup means for each)

• **Exchange Online mailboxes:** stale user and shared mailboxes; **inactive mailboxes** (retained after the

user was deleted); oversized archives. Retention/delete policies and labels apply here.

• **SharePoint Online sites:** dormant sites (no content change / no views for a long period), orphaned

project/team sites, oversized document libraries.

• **OneDrive for Business:** unused OneDrives (often belonging to departed users), duplicate/obsolete personal

files.

• **Microsoft Teams (chats & channel messages) and Microsoft 365 Groups:** finished-project teams, expired

event teams, and the group-connected SharePoint/mailbox behind them.

- **Data estate / storage reporting:** the cross-workload storage and activity picture that sizes the ROT and

measures savings. *Deeper data-estate scanning (e.g. Data Map / non-M365 sources) is licensing-dependent — verify your tenant.*

The identify→clean-up-defensibly journey (the core discipline)

> ***Report*** *stale/inactive/legacy data and existing retention + holds →* ***design*** *the labels/policies and the scope (static or adaptive) with the records function →* ***configure*** *labels and policies, auto-apply in* ***SIMULATION*** *→* ***validate*** *that scope is correct and* ***no held item would be deleted*** *→* ***pilot*** *on a* ***limited scope*** *with* `-WhatIf` *deletion previews and* ***disposition review*** *→* ***expand*** *to the intended scope under change control →* ***operate*** *(recurring reporting, disposition review, adaptive-scope refresh).*

What you can control

- Which **retention labels** exist and their action — **retain**, **retain-then-delete**, or **delete-only** —

and whether a label requires **disposition review** before deletion. *(Retention periods and which content they apply to are records determinations.)*

- Which **retention/deletion policies** cover which **locations**, and whether membership is a **static scope**

or an **adaptive scope** query.

- Which content is **auto-labeled**, and whether the auto-apply policy runs in **simulation** or is turned on.
- The **pilot scope** (limited sites/mailboxes) enforced first and the dwell/observation time before expansion.
- Who sits on the **disposition review board** and what the review workflow is.

Limitations (be honest with buyers)

- **Deletion is destructive.** Once a retention/delete action permanently removes content past its retention,

it is gone. This solution's discipline (limited-scope pilot, `-WhatIf`, hold checks, disposition review) exists because the failure mode is data loss, not a help-desk ticket.

- **Feature availability is licensing-/plan-dependent.** Basic retention policies and labels are broadly

available; **auto-apply based on classifiers/query, adaptive scopes, records management, and disposition review** typically require **Microsoft 365 E5** or the **E5 Compliance** add-on (some capabilities are in **preview**). Availability and portal paths **evolve** — this solution flags "verify your licensing/tenant" rather than asserting a feature is present.

- **Retention actions have latency.** Applying labels, timer-driven deletion, and disposition processing are

not instant — policies can take time to reach all locations and the deletion timer runs on Microsoft's cadence. "Nothing happened yet" is usually latency, not failure.

• **The retention schedule and the deletion authorization are not ours to set.** *How long* to keep each class,

what is under legal hold, and *whether* a deletion is defensible are **records/compliance/legal** determinations. This solution provides the method and the evidence; the determination is deferred.

Naming convention (used in every doc and script)

Every artifact this solution creates uses `DLC – <Purpose> – <Scope/Stage>`, reused verbatim across the docs, scripts, and the Purview portal. The standard set:

Artifact	Name
Retention label — retain then delete	`DLC – Retain-Delete – GeneralContent`
Retention label — delete-only (stale cleanup)	`DLC – Delete-Stale – LegacyContent`
Retention label — mandatory disposition review before delete	`DLC – Disposition Review – LegacyRecords`
Label publishing policy (pilot ring)	`DLC – Publish Cleanup Labels – Pilot`
Auto-apply retention-label policy (simulation)	`DLC – Auto-Apply Cleanup – Simulation`
Retention/deletion policy — pilot scope	`DLC – Delete Inactive Content – Pilot`
Adaptive scope — stale SharePoint sites	`DLC – Stale Sites – Adaptive Scope`
Adaptive scope — inactive mailboxes / departed users	`DLC – Inactive Mailboxes – Adaptive Scope`
Pilot scope (limited sites/users)	`DLC Cleanup Pilot – Stage1`
Disposition review board	`DLC Disposition Review – Board`

The standard deployment model

> ***Assess*** *current storage/ROT posture, existing retention, and holds →* ***Design*** *labels, policies, and scope (static/adaptive) with the records function →* ***Configure*** *labels and policies, auto-apply* ***in simulation*** *→* ***Pilot*** *on a* ***limited scope*** *with* `-WhatIf` *deletion previews and disposition review →* ***Validate*** *against pass/fail gates (correct scope,* ***no held item deletable****, savings measured, disposition evidence produced) →* ***Expand*** *to the intended scope and ongoing operations. Every high-impact change is piloted first, hold-checked, and reversible up to the point of permanent deletion — with a documented rollback.*

2. Requirements — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

These are the **solution-specific** prerequisites for identifying ROT / inactive / legacy data and defensibly cleaning it up with Microsoft Purview Data Lifecycle Management and Records Management. They build on — and do not repeat — the six shared prerequisites in `global-requirements-purview.md` (tenant + compliance licensing, least-privilege Purview roles, S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope + change control). Verify each

item below before starting the Implementation Guide.

For every requirement: **Description** (what it is), **Why required**, **How to verify**, **How to implement if missing**.

> ***E3 baseline vs E5 — verify your licensing.*** *Basic **retention policies** and **retention labels** (manual apply, static scope) are available on **Microsoft 365 E3**. The capabilities that make *targeted, automated, reviewable* cleanup possible — **auto-apply based on trainable classifiers / query / sensitive info type**, **adaptive scopes**, **Records Management**, and **disposition review** — generally require **Microsoft 365 E5** or the **E5 Compliance** add-on, and some are in **preview**. Where a step below needs an E5-class capability it is flagged **"verify your licensing/tenant."***

2.1 Licensing sufficient for the lifecycle features you will use (E3 baseline vs E5)

- **Description:** Confirm your plan covers the specific DLM/Records features this cleanup uses. **E3** covers

manual retention labels and basic retention/deletion policies with **static** scopes. **E5 / E5 Compliance** (or the standalone Records Management / Information Protection & Governance add-ons) is generally required for **adaptive scopes**, **auto-apply label policies** (classifier/query/SIT-based, including **simulation mode**), **Records Management**, and **disposition review**. This extends GR-#1 with the lifecycle-specific plan detail.

- **Why required:** A policy that relies on an adaptive scope, auto-apply, or disposition review your plan does

not include will not be creatable or will not function — and you may fall back to a manual/static approach.

- **How to verify:** In the **Microsoft 365 admin center** confirm the subscription/add-ons (E3 / E5 / E5

Compliance); in the **Purview portal** → **Data Lifecycle Management** and **Records Management** confirm the **Adaptive scopes**, **Auto-apply a label**, and **Disposition** areas are visible. **Flag "verify your licensing/tenant"** — availability evolves.

- **How to implement if missing:** Acquire/assign the required licensing before building adaptive/auto-apply/

disposition controls; otherwise scope the first wave to **manual labels + static-scope retention policies** (E3-capable) and defer the E5 capabilities.

2.2 Data-lifecycle / records least-privilege roles

- **Description:** The Purview **role groups** for lifecycle work — **Records Management** (create/manage

retention labels, label policies, file plan, and **disposition**), **Compliance Administrator** or **Compliance Data Administrator** (retention policies), **Retention Management** where present, and **Disposition Management** / the **Disposition Reviewers** membership for the review step. **View-Only** equivalents for read-only assessment. Global Administrator only where genuinely required. Extends GR-#2. Reading **inactive mailboxes** and mailbox statistics also needs Exchange Online recipient-read roles.

- **Why required:** Purview enforces fine-grained role groups; creating labels/policies, running disposition

review, and enumerating inactive mailboxes each require the matching role, and least privilege keeps a destructive capability tightly held.

- **How to verify: Purview portal** → **Settings** → **Roles & scopes** → **Permissions** (Microsoft Purview roles) —

confirm membership in Records Management / Compliance roles and the Disposition Reviewers group; in **Exchange admin** confirm the account can read recipients/inactive mailboxes.

- **How to implement if missing:** Assign the least-privilege role group under change control; keep **who can

author a delete policy **and** who can approve a disposition** separated where possible.

2.3 A storage / ROT baseline and the signals to compute it

- **Description:** The **data and reports** needed to size ROT and find stale content: **existing retention

policies, labels, and label policies**; the** holds **in force;** inactive-mailbox inventory **(**`Get-Mailbox -InactiveMailboxOnly`**);** mailbox last-logon / size **(Exchange mailbox statistics); and** SharePoint/OneDrive site last-activity and storage **via** Microsoft Graph usage reports** (`getSharePointSiteUsageDetail`, `getOneDriveUsageAccountDetail`, `getMailboxUsageDetail`). Extends GR-#4.

- **Why required:** Cleanup targets must be *identified and sized* from evidence, not guessed. Without the

activity/storage signals you cannot name a stale-site or inactive-mailbox candidate list or measure savings.

- **How to verify:** Confirm you can run the delivered **Discovery** and **Report** scripts (Graph

`Reports.Read.All` and Exchange/S&C read roles) and that **audit logging / usage reports** are enabled in the tenant.

- **How to implement if missing:** Enable the Graph reports permission and audit logging; use this solution's

Discovery/Report scripts to build the baseline before designing any policy.

2.4 An inventory of existing holds (litigation, eDiscovery case, retention) — the safety gate

- **Description:** A **complete enumeration of every hold** that could protect content from deletion:

litigation holds (`Get-Mailbox | … LitigationHoldEnabled`), **In-Place / eDiscovery case holds** (`Get-CaseHoldPolicy`), and **retain-action retention policies** (`Get-RetentionCompliancePolicy`). Any content within a hold's scope is **protected from purge** by Purview's retention principles.

- **Why required:** This is the **defensibility gate**. A delete policy must not be assumed to remove held

content, and — more importantly — you must be able to **prove** no in-scope item is under hold before enforcing. Deleting something later found to be under legal hold is a serious failure; the platform blocks the purge, but you still validate and evidence it.

- **How to verify:** Run the **Discovery** and **Validate** scripts to list all holds and confirm overlap with

your cleanup scope; confirm with the legal/eDiscovery function that the hold inventory is complete.

- **How to implement if missing:** Enumerate all holds first; if hold state is uncertain, **do not enforce any

deletion** — resolve the hold question with legal before proceeding.

2.5 A records / retention-schedule decision source (deferred, but required as input)

- **Description:** The **records-management decision** for each in-scope data class — *how long* it must be kept,

whether it is a **record** (immutable), and whether deletion requires **disposition review**. This is a records/compliance/legal determination (**deferred**), but the configuration cannot proceed without it as an input. Extends GR-#5.

- **Why required:** A retention label's period and delete action *encode* a retention decision. Setting them

without the records schedule means the tool is inventing a legal retention period — which this solution must never do.

- **How to verify:** Confirm the applicable **retention schedule / file plan** and the named approver(s) for

each data class in scope; confirm which classes require disposition review.

- **How to implement if missing:** Capture the schedule from the records/compliance function before authoring

labels; where a class's schedule is undecided, treat it as **retain / do-not-delete** until decided.

2.6 A limited pilot scope, adaptive-scope attribute data, and change control

- **Description:** A named **limited pilot scope** (`DLC Cleanup Pilot – Stage1` — a small set of pilot

sites/mailboxes that are known to be safe to act on, ideally non-production or clearly-obsolete), the **directory attributes** an adaptive scope will query (e.g. `Department`, account-enabled state, or a custom attribute marking a site/user/mailbox as decommissioned), and the **change-control window** to move from pilot to expansion. Extends GR-#6.

- **Why required:** Deletion is destructive, so it must be proven on a **small, safe scope first**. Adaptive

scopes need reliable attribute data or they target the wrong objects; a query that mis-selects is how a cleanup deletes live content.

- **How to verify:** Confirm the pilot scope membership, that the attributes an adaptive scope needs are

populated and accurate, and the approval path to expand.

- **How to implement if missing:** Define the pilot scope and validate/populate the adaptive-scope attributes

before building any adaptive policy; agree change control before enforcing deletion anywhere.

2.7 A disposition review board and workflow (where deletion needs a human gate)

• **Description:** A named **disposition review board** (`DLC Disposition Review – Board`) — the reviewers who,

for labels configured with **mandatory disposition review**, **approve, extend, or relabel** content reaching end-of-life — plus the workflow and evidence-retention expectation. Disposition review is an **E5 / Records Management** capability (verify licensing). Reviewers need the **Disposition Reviewers** role membership.

• **Why required:** For anything sensitive or of record value, permanent deletion should have a **human

approval step **that produces** auditable disposition evidence** (who approved deleting what, when) — the backbone of a defensible cleanup.

• **How to verify:** Confirm the reviewer membership, that labels requiring review are configured with

disposition review = on, and that **Purview portal** → **Records Management** → **Disposition** is available in your tenant.

• **How to implement if missing:** Where disposition review is not licensed, restrict E5-gated auto-cleanup and

use **manual, evidenced disposition** on a limited scope; assign reviewers and define the workflow before enabling delete labels on record-value content.

Requirements summary

#	Requirement	Gate
GR 1–6	Category prerequisites (`global-requirements-purview.md`)	Tenant/licensing, roles, toolchain, scope, stakeholders, pilot + change control
2.1	Lifecycle licensing (E3 baseline vs E5 for adaptive/auto-apply/disposition; verify)	Needed features available
2.2	Data-lifecycle / records least-privilege roles	Author + reviewer roles assigned, separated
2.3	Storage / ROT baseline + signals (Graph usage reports, mailbox stats)	Candidates identified and sized
2.4	Complete inventory of holds (litigation, eDiscovery case, retention)	**No held item deletable — proven**
2.5	Records / retention-schedule decision source (deferred input)	Labels encode a real schedule, not a guess
2.6	Limited pilot scope + adaptive-scope attribute data + change control	Deletion proven on a safe small scope first
2.7	Disposition review board + workflow (where required)	Human gate + disposition evidence

> ***Deferred to records / compliance / privacy / legal:*** *the retention schedule (how long to keep each class), whether content is a record, what is under legal hold, and whether a given deletion is defensible. This solution identifies candidates, configures the control, proves no held item would be deleted, and produces the disposition evidence — it does not make the retention or deletion determination.*

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

This is the exact, ordered procedure to **identify** ROT / inactive / legacy data and **defensibly clean it up** with Microsoft Purview — auto-apply built in **SIMULATION** first, deletion **piloted on a limited scope** and **previewed with** `-WhatIf`, and **every hold checked and respected** before anything is enforced. Portal paths and cmdlets are from Microsoft's public documentation; **licensing-/plan-dependent features and evolving portal paths are flagged "verify your licensing/tenant"** and are never asserted for your tenant. Do the phases in order. Each phase names its artifacts (from the Overview naming table), states a **pass/fail gate**, and gives a **rollback**.

> ***Automation.*** *The five delivered PowerShell tools run through **Security & Compliance PowerShell** (`Connect-IPPSSession`), **Exchange Online PowerShell** (inactive mailboxes / statistics), and **Microsoft Graph** (usage reports). They are **read-only or** `-WhatIf` **by default**; writes require an explicit `-Commit` switch. **None of them permanently deletes content** — the destructive action stays a deliberate, hold-checked portal/policy step under change control. No credentials are stored in code.*
>
> ***Determinations deferred.*** **How long* to keep each class, *what* is under legal hold, whether content is a **record**, and whether a deletion is **defensible** are **records / compliance / privacy / legal** decisions (Req 2.4, 2.5). Capture the retention schedule and the confirmed hold inventory **before** Phase 3, and the deletion authorization **before** Phase 5.*

Phase 0 — Connect and baseline the storage / ROT posture (Assess)

1. **Connect the toolchain** (GR-#3). On the admin workstation:

```
Install-Module ExchangeOnlineManagement -Scope CurrentUser -Force   # Connect-IPPSSession + Connect-Exc
Connect-IPPSSession   -UserPrincipalName admin@contoso.com          # Security & Compliance PowerShell
Connect-ExchangeOnline -UserPrincipalName admin@contoso.com         # inactive mailboxes / mailbox stat
Connect-MgGraph -Scopes 'Reports.Read.All','Organization.Read.All' -NoWelcome   # Graph usage reports
```

1. **Run the posture assessment** (delivered script #1, read-only):

```
.\DLCCLEAN-Discovery.ps1 -CsvPath .\DLCCLEAN-Posture.csv
```

It reports existing **retention policies/labels/label policies** and their scope, **all holds in force** (litigation, eDiscovery case, retention), the **inactive-mailbox inventory**, and mailbox/site/OneDrive **last-activity + storage** signals — the ROT candidate baseline.

1. **Build the sized ROT candidate list** (delivered script #4, read-only):

```
.\DLCCLEAN-Report.ps1 -StaleDays 365 -CsvPath .\DLCCLEAN-ROT-Candidates.csv
```

It lists stale sites/OneDrives/mailboxes (no activity beyond `-StaleDays`) with storage consumed, so the records/storage owners see named, sized candidates — not a guess.

1. **Confirm licensing** (Req 2.1): confirm E5/E5-Compliance where adaptive scopes, auto-apply, or disposition

review are needed; if E3-only, plan a manual-label + static-scope first wave.

Pass/fail gate: you have a written baseline (existing retention + **complete hold inventory**), a **sized ROT candidate list**, and confirmed licensing for the capabilities you will use. **Rollback:** none (read-only).

Phase 1 — Design the labels, policies, and scope (Design — with the records function)

Decide, **per in-scope data class**, the retention action and the scope. Do **not** invent retention periods — they come from the records schedule (Req 2.5). *Whether adaptive scopes / auto-apply / disposition review are available is licensing-dependent — verify your tenant.*

1. **Create the retention labels** you need in **Purview portal** → **Solutions** → **Records management** → **File plan**

(or **Data lifecycle management** → **Labels**):

• `DLC – Retain-Delete – GeneralContent` — **retain then delete** after the records-defined period.

• `DLC – Delete-Stale – LegacyContent` — **delete-only** (no retain) for content the schedule says may simply

be removed once stale.

• `DLC – Disposition Review – LegacyRecords` — **retain then delete with mandatory disposition review** for

record-value content (requires Records Management / E5 — verify). *The period and which content each label applies to are the records function's determination.*

1. **Design the scope for each cleanup policy** — **static** (named sites/mailboxes) for the pilot, **adaptive**

for ongoing targeting where licensed:

• `DLC – Stale Sites – Adaptive Scope` — a **SharePoint site** adaptive scope querying the attribute(s) that

mark a site stale/decommissioned (Req 2.6).

• `DLC – Inactive Mailboxes – Adaptive Scope` — a **User** adaptive scope for departed/disabled users'

mailboxes. Portal: **Data lifecycle management** → **Adaptive scopes** → **Create scope**. *Adaptive scopes require E5-class licensing — verify.*

1. **Decide the auto-apply conditions** (if using auto-apply): which content should receive

`DLC – Retain-Delete – GeneralContent` automatically (by SIT, keyword/query, or trainable classifier) — this will run in **simulation** first (Phase 2).

Pass/fail gate: every in-scope data class maps to a named label with a **records-approved** action/period, and each cleanup target maps to a named static or adaptive scope; disposition-review labels are identified. **Rollback:** delete unused labels/scopes (they have no effect until published/applied by a policy).

Phase 2 — Configure and publish/auto-apply in SIMULATION (Configure — no deletion)

1. **Publish the labels to the pilot scope** — `DLC – Publish Cleanup Labels – Pilot`.

Portal: **Data lifecycle management** → **Label policies** → **Publish labels** → select the DLC labels → **scope: the limited pilot locations only** (`DLC Cleanup Pilot – Stage1`). Publishing only *makes labels available / manually appliable*; it deletes nothing.

1. **Create the auto-apply policy in simulation** — `DLC – Auto-Apply Cleanup – Simulation` (E5 — verify).

Portal: **Data lifecycle management** → **Label policies** → **Auto-apply a label** → choose `DLC – Retain-Delete – GeneralContent` → set the condition (SIT/keyword/query/classifier) → **scope: pilot** → **choose "Run policy in simulation mode."** Simulation shows *what would be labeled* with **no** label actually applied.

1. **Create the deletion/retention policy in PILOT, limited scope** — `DLC – Delete Inactive Content – Pilot`.

Verify (read-only preview) with the config helper first, then commit to the **pilot scope only**:

```
# Preview only — no change:
.\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `
    -Workload SharePoint -RetentionAction Delete -RetentionDays 2555 `
    -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -WhatIf
# Create it, still scoped to the limited pilot location:
.\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `
    -Workload SharePoint -RetentionAction Delete -RetentionDays 2555 `
    -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -Commit
```

The helper creates the policy on the **named pilot location only** and never applies it tenant-wide; the retention period comes from the records schedule (Req 2.5), not the tool.

Pass/fail gate: labels published to pilot; auto-apply policy confirmed **in simulation** (no labels applied); the deletion policy exists but is **scoped to the limited pilot location only**. **Rollback:** unpublish the label policy / delete the simulation and pilot policies — no content was deleted, so removal is clean.

Phase 3 — Validate scope and prove no held item is deletable (Validate — the safety gate)

This phase is the **defensibility gate** and must pass before any expansion.

1. **Validate the pilot policy** (delivered script #3, read-only):

```
.\DLCCLEAN-Validate.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -CsvPath .\DLCCLEAN-Valida
```

It confirms: the policy is scoped to the **intended pilot location(s) only** (not "All"); the **retention action/period** matches the design; and — critically — it **cross-checks every hold** (litigation, eDiscovery case, retain-action retention) against the policy scope and **flags any overlap** where an item could be under hold. **PASS = scope correct AND no in-scope item under hold would be purged.**

1. **Review the auto-apply simulation results.** Portal: open `DLC – Auto-Apply Cleanup – Simulation` → the

simulation results show exactly which items *would* be labeled. Confirm they are the intended stale/legacy content and not live business data.

1. **Confirm the hold inventory with legal/eDiscovery** (Req 2.4). If any hold state is uncertain, **stop** —

do not proceed to enforcement until legal confirms.

Pass/fail gate: scope verified limited-and-correct; **hold cross-check clean** (no deletable held item); auto-apply simulation shows only intended content. **Rollback:** none needed (read-only validation); if the check fails, **narrow the scope / exclude the held locations / resolve the hold** before continuing — do not enforce.

Phase 4 — Pilot the cleanup with `-WhatIf` and disposition review (Pilot)

1. **Preview the deletion impact on the pilot scope** (script #4, read-only):

```
.\DLCCLEAN-Report.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -Mode DispositionPreview `
    -CsvPath .\DLCCLEAN-Disposition-Preview.csv
```

This lists the items the pilot policy would act on and the storage they represent — the disposition-review input. **No deletion occurs.**

1. **Turn on auto-apply for the pilot** only after simulation is satisfactory: in the portal, switch

`DLC – Auto-Apply Cleanup – Simulation` from **simulation** to **on** — *for the pilot scope only*. Labels now apply to pilot content (retain/retain-then-delete), still no immediate deletion (deletion happens at end-of-retention, gated by disposition review where configured).

1. **Run disposition review** for `DLC – Disposition Review – LegacyRecords` content reaching end-of-life.

Portal: **Records management** → **Disposition** → the `DLC Disposition Review – Board` reviewers **approve, extend, or relabel** each item. Approval is what authorizes permanent deletion — and produces the **disposition evidence** (who approved what, when). *(E5 / Records Management — verify.)*

1. **Confirm scope stayed limited.** Re-run `DLCCLEAN-Validate.ps1` and confirm nothing outside

`DLC Cleanup Pilot – Stage1` was affected and the hold cross-check is still clean.

Pass/fail gate: pilot content is labeled/dispositioned as designed; disposition reviewers produced approval evidence; **no content outside the pilot scope was touched**; hold cross-check remains clean. **Rollback:** remove the pilot policy / unpublish labels / set auto-apply back to simulation — reversible up to the point a disposition approval permanently deletes an item. Content already permanently deleted past retention is **not** recoverable — which is why the hold check and disposition review gate it.

Phase 5 — Expand under change control and operate (Expand → Operate)

1. **Expand the scope** from the limited pilot to the intended population **one workload/scope at a time**,

swapping the static pilot location for the `DLC – Stale Sites – Adaptive Scope` / `DLC – Inactive Mailboxes – Adaptive Scope` where licensed — **re-running** `DLCCLEAN-Validate.ps1` **and the hold cross-check after each expansion** (Req 2.6). Each expansion is a change-controlled step with the **deletion authorization** captured (Req 2.5).

1. **Extend to inactive mailboxes and orphaned Groups/Teams** deliberately: inactive mailboxes are already

retained by a hold — apply the records-approved retention/label so they are eventually dispositioned rather than kept forever; clean up finished-project Teams/Groups and their connected sites/mailboxes per the same gated process.

1. **Operationalize:** schedule `DLCCLEAN-Report.ps1` for a **recurring storage-savings and ROT report**, run

disposition review on cadence, and **refresh adaptive-scope attribute data** so membership stays accurate as the estate changes. Re-baseline periodically with `DLCCLEAN-Discovery.ps1`.

1. **Validate end-to-end** with the delivered testing procedure (next section) and record the pass/fail

evidence, including the storage-savings measurement and the disposition-approval trail.

Pass/fail gate: cleanup live across the intended scope with the records-approved action per class; **hold cross-check clean at every expansion**; disposition evidence captured; measured storage savings recorded. **Rollback:** any policy can be scoped back or removed; auto-apply can return to simulation; **but permanent deletion past retention is irreversible** — expansion only proceeds where the hold check is clean and the deletion is authorized.

Order-of-operations summary

Phase	Do	Artifact(s)	Gate	Rollback
0 Assess	Connect, baseline retention+holds, size ROT	Posture + ROT-candidate CSV	Baseline + hold inventory + sized candidates	n/a (read-only)
1 Design	Labels, policies, static/adaptive scope with records fn	DLC labels + adaptive scopes	Every class → records-approved label + named scope	Delete unused label/scope
2 Configure	Publish labels; auto-apply **simulation**; delete policy **pilot scope only**	Sim + pilot policies	Sim confirmed, delete policy limited-scoped	Unpublish / delete (nothing deleted)

Phase	Do	Artifact(s)	Gate	Rollback
3 Validate	Verify scope; **prove no held item deletable**	Validation CSV	Scope correct + **hold cross-check clean**	Narrow scope / resolve hold (don't enforce)
4 Pilot	`-WhatIf` preview; auto-apply on for pilot; disposition review	Disposition preview + evidence	Pilot only; disposition approved; holds clean	Back to simulation / remove (pre-purge)
5 Expand	Broaden scope 1-at-a-time; adaptive; operate	Enforced policies	Full scope, holds clean, savings measured	Scope back (purge is irreversible)

5. Testing & Validation — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

Validate the cleanup implementation against explicit **pass/fail gates**. The four headline gates are: **(A) the right stale/legacy data is identified and scoped**, **(B) no held item is deletable — the safety gate**, **(C) cleanup acts only where intended and produces disposition evidence**, and **(D) savings are measured and the control is reversible up to permanent deletion**. Run every test with **fabricated / clearly non-production data or a designated safe pilot scope** — **never** by deleting real business content to "see if it works." Record the evidence.

> *All tests assume the artifacts from the Implementation Guide: labels published to the pilot, auto-apply in **simulation**, and the deletion policy scoped to `DLC Cleanup Pilot – Stage1` only. Deletion is destructive — **the hold cross-check (Gate B) must pass before any enforcement.***

Test A — Identification & scoping (the right data is targeted)

#	Test	Steps	Pass condition
A1	ROT candidates sized	Run `DLCCLEAN-Report.ps1 -StaleDays 365 -CsvPath ...`	A named, sized list of stale sites/OneDrives/mailboxes (last-activity + storage) is produced.
A2	Inactive mailboxes enumerated	Run `DLCCLEAN-Discovery.ps1`; inspect the inactive-mailbox section.	Inactive mailboxes (`Get-Mailbox -InactiveMailboxOnly`) are listed with the hold that retains them.
A3	Auto-apply simulation accuracy	Open `DLC – Auto-Apply Cleanup – Simulation` results.	The items that *would* be labeled are the intended stale/legacy content — no live business data.

#	Test	Steps	Pass condition
A4	Scope is limited, not "All"	Run `DLCCLEAN-Validate.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot'`.	Policy scope is the named pilot location(s) only — **not** an "All" / tenant-wide location.
A5	Adaptive scope precision (if used)	Review the members an adaptive scope resolves to.	Members match the intended stale/decommissioned attribute query — no live/active objects mis-selected.

Gate A (pass): stale/legacy candidates are identified and sized; the deletion policy is confirmed **limited-scoped** to the intended targets.

Test B — Hold safety (no held item is deletable) — the defensibility gate

#	Test	Steps	Pass condition
B1	Hold inventory complete	`DLCCLEAN-Discovery.ps1` lists litigation, eDiscovery case, and retention holds.	Every hold class is enumerated; the count matches what legal/eDiscovery confirms (Req 2.4).
B2	Scope vs hold cross-check	`DLCCLEAN-Validate.ps1` cross-checks the delete-policy scope against all holds.	**No in-scope location/mailbox/site is under any hold** — or every overlap is flagged and excluded. **PASS = zero deletable held items.**
B3	Held content survives a delete action	On a fabricated item placed under a test hold within (a copy of) the scope, confirm a delete label/policy does **not** purge it.	Held content is **retained** despite the delete action (retention principles: hold blocks purge).
B4	Uncertain hold halts enforcement	Simulate an unconfirmed hold state.	The process **stops** — enforcement does not proceed while any hold state is unconfirmed.

Gate B (pass): the hold cross-check is **clean** (no deletable held item) and any overlap is excluded or resolved with legal before enforcement. **This gate is mandatory before Phase 4/5.**

Test C — Controlled cleanup & disposition evidence (acts only where intended)

#	Test	Steps	Pass condition
C1	`-WhatIf` preview makes no change	Run the config/report tools without `-Commit`.	Only a preview of what *would* happen is produced; nothing is created, labeled, or deleted.
C2	Pilot-only action	With auto-apply on for the pilot, confirm content **outside** `DLC Cleanup Pilot – Stage1` is untouched.	No labeling/deletion occurs outside the pilot scope.
C3	Disposition review gate	For `DLC – Disposition Review – LegacyRecords` content at end-of-life, run the review.	Reviewers must **approve** before permanent deletion; approval/extension/relabel is recorded.
C4	Disposition evidence captured	Inspect the disposition record for a reviewed item.	An auditable record exists: who approved deleting what, and when (defensible-cleanup evidence).
C5	Retention wins where classes overlap	Apply a retain label and a delete label to overlapping content.	The **retain** action wins (retention precedence) — content is not prematurely deleted.

Gate C (pass): cleanup acts **only** on the pilot scope, deletion of record-value content passes through disposition review, and the disposition evidence is captured.

Test D — Savings measurement & reversibility

#	Test	Steps	Pass condition
D1	Storage baseline vs projected savings	Compare the `DLCCLEAN-Report.ps1` ROT-candidate storage before, and the projected/actual reclaimed storage after.	A measured storage figure exists (candidate GB identified; reclaimed GB after disposition).

#	Test	Steps	Pass condition
D2	Reversible pre-purge	Set auto-apply back to **simulation** / remove the pilot policy before any permanent deletion.	The control is removed with **no content lost** (reversible up to permanent deletion).
D3	Scripts safe-by-default	Run `DLCCLEAN-Config.ps1` and `DLCCLEAN-Set-Scope.ps1` **without** `-Commit`.	They make no change (`-WhatIf` output only); Discovery/Validate/Report are read-only.
D4	No tool deletes content	Review all five scripts.	**None** permanently deletes content — deletion remains a hold-checked portal/policy action.

Gate D (pass): savings are measured, the control is reversible before permanent deletion, and every tool is confirmed non-destructive.

Evidence to capture (for the deliverable record)

- Baseline posture CSV + **complete hold inventory** (Phase 0 / Test B1).
- Sized **ROT candidate list** and the measured storage figure (Test A1 / D1).
- The **hold cross-check result** proving no deletable held item (Test B2) — the defensibility centerpiece.
- Auto-apply **simulation results** and the `-WhatIf` disposition preview (Test A3 / C1).
- The **disposition-approval trail** (who approved deleting what, when) for reviewed content (Test C3/C4).
- Confirmation that action stayed inside the pilot scope (C2) and that the control reverts pre-purge (D2).

> ***Determination boundary.*** *These tests prove the* ***control works and is defensible****. Whether a given class's retention period, its records status, what is under legal hold, and whether a deletion is *authorized* are* ***records / compliance / legal*** *sign-offs, captured alongside the evidence above.*

6. Troubleshooting — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

Symptom-first fixes for the failures you actually hit identifying ROT and cleaning up inactive/legacy data with Purview Data Lifecycle Management. Most problems are one of: nothing happens (latency), the wrong content is targeted, held content is unexpectedly in scope, an adaptive scope mis-selects, a label/policy won't apply, or a report comes back empty. **Remember the golden rule:** if a hold check is failing or uncertain, **stop and do not enforce deletion** — troubleshoot before, not after, content is removed.

Nothing seems to happen

I applied a delete policy/label and nothing was deleted.

- **This is usually latency, not failure.** Retention label application, the deletion timer, and disposition

processing run on **Microsoft's cadence**, not instantly — policies can take time to reach all locations and the timer runs periodically. Allow dwell time before concluding it is broken.

- Confirm the policy is **Enabled** — the config helper deliberately creates the pilot policy **disabled**

(`Enabled=$false`) so you validate holds first. It does nothing until you enable it under change control.

- Confirm the content is actually **past the retention period** — a "delete after 2555 days" rule does nothing

to content that is younger.

- **A hold is (correctly) blocking the purge.** If the content is under any hold, the delete action will **not**

remove it — that is the platform protecting held data (see below). Check with `DLCCLEAN-Validate.ps1`.

Auto-apply simulation shows matches but no labels were applied.

- That is **correct** — simulation shows what *would* be labeled with nothing applied. To apply, switch the

auto-apply policy from **simulation** to **on** for the pilot scope (Implementation Phase 4).

Wrong content targeted / too much in scope

The policy is scoped to "All" / tenant-wide when I wanted a pilot.

- `DLCCLEAN-Validate.ps1` **Gate 1** flags this. Narrow the scope with `DLCCLEAN-Set-Scope.ps1`

(`-RemoveLocation`, add a named site/mailbox, or bind an adaptive scope). The config and set-scope helpers **refuse to add "All"** by design.

- Never pilot a delete action on "All" — the whole discipline is a **limited scope first**.

An adaptive scope is selecting live/active objects, not just stale ones.

- The adaptive query is matching on **attributes that are wrong or unpopulated** (Req 2.6). Inspect what the

scope resolves to **before** binding a delete policy to it, and **refresh the attribute data**. A query over bad attributes mis-targets — fix the attributes, not the policy.

- Prefer a **static** named scope for the pilot; move to adaptive only once membership is proven accurate.

Content I expected to be cleaned up is being kept instead.

- **Retention precedence:** *retain wins over delete*, and the *longest* retention wins. Another retain

label/policy (or a hold) in scope is keeping it. `DLCCLEAN-Validate.ps1` Gate 3 lists the overlapping retain policies/holds. This is by design — resolve it with the records function, don't force-delete.

Held content is unexpectedly in scope (the safety gate)

`DLCCLEAN-Validate.ps1` **Gate 3 (hold cross-check) reports FAIL / overlaps.**

• **Do not enforce.** An in-scope location overlaps a litigation hold, an eDiscovery case hold, or a retain

policy. Either **exclude** those locations from the cleanup scope (`DLCCLEAN-Set-Scope.ps1 -RemoveLocation`) or **resolve the hold** with legal/eDiscovery — then re-run Validate until Gate 3 is **PASS**.

• The platform will still block the purge of held items, but the point is to **prove** it before acting — a

clean cross-check is your defensibility evidence.

The hold inventory looks incomplete.

• Litigation-hold enumeration needs **Exchange Online** connected (`Connect-ExchangeOnline`) — if it is not,

`DLCCLEAN-Discovery.ps1` / `-DLCCLEAN-Validate.ps1` say so and skip that class. Connect it for a complete check.

• eDiscovery **case holds** need `Get-CaseHoldPolicy` (Records/eDiscovery role). Confirm your role and re-run.

• If any hold state is **uncertain**, treat it as held and **stop** until legal confirms (Req 2.4).

Labels / policies won't create or apply

Adaptive scope / auto-apply / disposition review isn't available in the portal.

• These are **E5 / E5 Compliance** (or Records Management) capabilities and some are **preview** — **verify

your licensing/tenant **(Req 2.1). On** E3** you have manual labels + static-scope retention only; plan the first wave accordingly.

`New-RetentionCompliancePolicy` **/** `New-ComplianceTag` **fails or the cmdlet isn't recognized.**

• Ensure the **ExchangeOnlineManagement** module is installed/updated and you connected with

`Connect-IPPSSession` (Security & Compliance PowerShell), not plain Exchange Online.

• Confirm the account holds the required **Records Management / Compliance Administrator** role — a missing role

surfaces as "cmdlet not recognized" or access denied (Req 2.2).

A retention label won't apply to content even though it's published.

• **Publishing latency** — a newly published label can take time to appear where users/policies can apply it.

• Confirm the **label policy scope** includes the location you are testing, and that the label's action/period

is what you intend.

Reports / discovery come back empty

`DLCCLEAN-Report.ps1` **returns no stale candidates.**

• **Graph usage reports** need `Reports.Read.All` — connect with

`Connect-MgGraph -Scopes 'Reports.Read.All'` and re-run.

• **Reports may be de-identified.** If tenant privacy settings **conceal user/site names** in usage reports,

the identity columns come back anonymized — toggle the setting in the M365 admin center (reports privacy) if you need named candidates, under your privacy policy.

• Widen `-StaleDays` — if the window is too short nothing qualifies as stale.

`Get-Mailbox -InactiveMailboxOnly` **returns nothing.**

• There may genuinely be **no inactive mailboxes** (an inactive mailbox exists only when a mailbox was placed

on hold and *then* the user was deleted). Confirm with the messaging team.

• Confirm **Exchange Online** is connected and your account can read recipients (Req 2.2).

Something changed unexpectedly

A script made a change I didn't expect.

• The delivered scripts are **read-only or** `-WhatIf` **by default**; writes require `-Commit`, and **none

permanently deletes content**. The config helper creates pilot policies** disabled**; the set-scope helper changes** scope only** (never the action or enabled state). If a change landed, it was an explicit `-Commit`.

• The safe recovery is always to **disable / narrow the policy** (`DLCCLEAN-Set-Scope.ps1`) and **re-validate**

— nothing is deleted until an enabled policy's timer runs on unheld, past-retention content.

Fast triage table

Symptom	First check
Nothing deleted	Latency (Microsoft timer); policy still **disabled**; content not past retention; a hold is blocking (correct)
Simulation matched but no labels applied	Auto-apply still in **simulation** — that's expected; turn on for pilot
Scope is "All"	Validate Gate 1 — narrow with Set-Scope; tools refuse to add "All"
Adaptive scope mis-selects	Wrong/unpopulated attributes — inspect resolved members, refresh attributes, prefer static for pilot
Expected-cleanup content kept	Retention precedence — a retain label/policy or hold wins (Validate Gate 3)
Hold cross-check FAIL	**Do not enforce** — exclude the held locations or resolve the hold, re-validate to PASS
Hold inventory incomplete	Connect Exchange Online (litigation) + Records/eDiscovery role (case holds)

Symptom	First check
Adaptive/auto-apply/disposition missing	E5 / E5 Compliance licensing — verify tenant
Cmdlet not recognized / denied	ExchangeOnlineManagement updated, `Connect-IPPSSession`, Records/Compliance role
Label won't apply	Publishing latency; label-policy scope; label action/period
Report empty	Graph `Reports.Read.All`; reports de-identified in tenant privacy; widen `-StaleDays`
No inactive mailboxes	May be none (hold-then-delete only); Exchange connected + recipient-read role
Unexpected change	Scripts are `-WhatIf`/read-only by default; none deletes; recover by disabling/narrowing + re-validate

7. Best Practices — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

Field-tested practices for identifying ROT and defensibly cleaning up inactive/legacy data so the control cuts storage and exposure **without ever destroying data that should have been kept**. These complement the Implementation and Testing guides. The golden rule: **deletion is irreversible — prove it safe before you enforce it, and never auto-delete.**

Check holds first, every time — the defensibility gate

• **Enumerate every hold** (litigation, eDiscovery case, retain-action retention) **before** designing any

delete policy, and **re-run the cross-check after every scope change** (`DLCCLEAN-Validate.ps1`). Purview's retention principles mean a hold *blocks* purge — but "the platform protects it" is not evidence; **prove** no in-scope item is held.

• If **any** hold state is uncertain, **stop**. Resolve it with legal/eDiscovery before enforcing. A deletion

later found to have targeted held content is a serious, sometimes legal, failure.

• Keep the **hold inventory and the cross-check result** as part of the deliverable evidence — it is what makes

the cleanup defensible.

Never auto-delete — pilot small, preview with `-WhatIf`, expand slowly

• **Nothing is auto-deleted by these tools.** The scripts are read-only or `-WhatIf`; the destructive action is

always a deliberate, hold-checked policy step under change control.

• **Pilot on a small, known-safe scope first** (`DLC Cleanup Pilot – Stage1`) — non-production or clearly-obsolete

content — before touching the wider estate. Expand **one workload/scope at a time**, re-validating each.

- **Preview impact before committing:** run `DLCCLEAN-Report.ps1 -Mode DispositionPreview` and the auto-apply

simulation so you see exactly what *would* be labeled/removed, and its storage, before it happens.

Use simulation and disposition review as human gates

- **Run auto-apply label policies in SIMULATION first** — they show what *would* be labeled with no label

applied. Read the results and confirm they are stale/legacy content, not live business data, before turning on.

- For record-value or sensitive content, require **mandatory disposition review** (`DLC – Disposition Review –

LegacyRecords`) so a human **approves** each permanent deletion and the approval is **evidenced**. Reserve delete-only labels for content the records schedule clearly permits to simply age out.

- **Separate who authors a delete policy from who approves a disposition** where possible (Req 2.2) — least

privilege on a destructive capability.

Understand retention precedence before you design

- Retention and deletion are **one engine** governed by the **principles of retention**: **retention wins over

deletion**, the** longest **retention wins,** explicit **beats** implicit**, and a** hold blocks purge**. Design cleanup knowing that a retain label/policy anywhere in scope will keep content your delete policy targets — that is a feature (it protects), not a bug.

- Don't stack overlapping delete policies hoping one "wins" — they don't purge held/retained content, and the

interaction gets hard to reason about. Prefer **fewer, well-scoped** labels and policies.

Get the scope right — static for pilot, adaptive for scale

- **Start static** (named pilot sites/mailboxes) so the pilot blast radius is exactly what you chose.
- Move to **adaptive scopes** for ongoing targeting (stale sites, departed-user mailboxes) **only once the

attribute data is reliable **(Req 2.6) — an adaptive query over bad attributes selects the wrong objects. Test what an adaptive scope resolves to** before** binding a delete policy to it.

- **Refresh adaptive-scope attribute data** on a cadence — membership drifts as the estate changes; stale

attributes silently mis-target.

Measure ROT and savings from evidence

• Size ROT from **real signals** — mailbox/site/OneDrive **last-activity and storage** (Graph usage reports),

inactive-mailbox inventory — not from assumptions. Name and size the candidates so the records/storage owners can decide.

• **Report savings** (candidate GB identified vs GB reclaimed after disposition) on a recurring cadence — it is

the business case and it shows the control is working.

• Re-baseline periodically (`DLCCLEAN-Discovery.ps1`) — new ROT accumulates continuously; cleanup is an ongoing

operation, not a one-time project.

Mind the platform's behavior

• **Retention actions have latency** — label application, the deletion timer, and disposition processing run on

Microsoft's cadence, not instantly. Allow dwell time before concluding a policy "didn't work."

• **Inactive mailboxes are already retained by a hold** — cleaning them up means applying a records-approved

retention/label so they are eventually dispositioned, not kept forever; removing the hold without a plan can make them permanently unrecoverable.

• **Verify licensing/tenant** before relying on adaptive scopes, auto-apply, records management, or disposition

review — several are E5/E5-Compliance (some preview), and portal paths evolve.

Stay inside the boundary

• This solution provides the **technical method and the evidence**. The **retention schedule (how long),

records status, legal-hold state, and whether a deletion is authorized/defensible are records / compliance / privacy / legal determinations** — you find the candidates, configure the control, prove no held item is deletable, and produce the disposition evidence; they own the decision to delete.

8. Deliverables Summary — Inactive & Legacy Data Cleanup & Storage Optimization (Microsoft Purview)

What you receive in this solution, and what "done" looks like.

Documentation

Document	What it gives you
Solution Overview	The identify→clean-up-defensibly model — ROT/inactive/legacy discovery, retention labels (retain / retain-then-delete / delete-only / disposition-review), retention & deletion policies, static vs adaptive scopes, the in-scope workloads (Exchange, SharePoint, OneDrive, Teams, Groups), and why deletion is hold-checked and never automatic; naming convention.
Requirements Guide	Solution-specific prerequisites (E3-vs-E5 lifecycle licensing, records/lifecycle roles, storage/ROT baseline signals, the **complete hold inventory**, the records-schedule decision source, limited pilot scope + adaptive-scope attributes, disposition review board) each with verify/implement steps, on top of the six category prerequisites.
Implementation Guide	The exact ordered phases: baseline retention+holds and size ROT → design labels/scopes with the records function → publish/auto-apply **in simulation** and pilot the delete policy on a **limited scope** → **validate no held item is deletable** → pilot with `-WhatIf` + disposition review → expand under change control; each phase named, gated, and with a rollback.
Testing & Validation	Pass/fail gates for the four headline outcomes — right data identified/scoped (A), **no held item deletable** (B), controlled cleanup + disposition evidence (C), savings measured + reversible (D) — all with fabricated/safe pilot data.
Troubleshooting Guide	The common failure modes (nothing deleted, wrong content targeted, held content unexpectedly in scope, adaptive scope mis-selecting, disposition/label not applying, reports empty) and how to resolve them.
Best Practices	Check-holds-first discipline, never-auto-delete, simulation + disposition-review gates, retention precedence, static-vs-adaptive scoping, ROT/savings measurement, and platform-latency realities.
Executive Summary	One-page business framing for approvers.

PowerShell tools (5) — Security & Compliance PowerShell / Exchange Online / Graph

All tools are **read-only or `-WhatIf` by default**; writes require an explicit `-Commit`. **None permanently deletes content** — deletion stays a hold-checked portal/policy action. No credentials are stored in code. They connect via `Connect-IPPSSession`, `Connect-ExchangeOnline`, and/or `Connect-MgGraph`.

#	Script	Role	Run mode / default safety
1	**DLCCLEAN-Discovery.ps1**	Storage/ROT & stale-data posture: existing retention policies/labels/label policies and their scope, **all holds in force** (litigation, eDiscovery case, retention), inactive-mailbox inventory, and mailbox/site/OneDrive last-activity + storage signals.	**Read-only**
2	**DLCCLEAN-Config.ps1**	Config helper: creates/updates a retention/deletion policy or label on a **named limited scope** (simulation/pilot first). Safe by default; never broad-enforces or auto-deletes.	`-WhatIf` (writes need `-Commit`; never deletes content)
3	**DLCCLEAN-Validate.ps1**	Validation checker: confirms the policy is scoped correctly (not "All"), the action/period match design, and — the safety gate — **cross-checks every hold** so no in-scope item under hold would be deleted. Per-gate pass/fail.	**Read-only**
4	**DLCCLEAN-Report.ps1**	Storage-savings / candidate-ROT / disposition report: sized stale-data candidates and a disposition preview for review.	**Read-only**

#	Script	Role	Run mode / default safety
5	**DLCCLEAN-Set-Scope.ps1**	Scope/health helper: reports a policy's locations/scope/mode; with `-Commit` adjusts included/excluded locations or binds an adaptive scope. Never changes the retain/delete action or deletes content.	`-WhatIf` (writes need `-Commit`)

Definition of done

• Storage/ROT baseline captured and the **complete hold inventory** enumerated; stale/inactive/legacy candidates

identified and sized.

• Retention labels mapped to every in-scope class with a **records-approved** action/period; cleanup targets

mapped to named static (pilot) or adaptive scopes.

• Auto-apply proven **in simulation**; the deletion policy piloted on a **limited scope** and previewed with

`-WhatIf`.

• **Hold cross-check clean** — proven that no in-scope item under hold would be deleted (Gate B) — before any

enforcement, and re-checked at every expansion.

• Cleanup acted **only** where intended; record-value deletions passed through **disposition review** with the

approval evidence captured (Gate C); measured **storage savings** recorded (Gate D).

• Safe-by-default, non-destructive scripts verified; change log and records/legal **deletion authorization**

captured.

> ***Boundary:*** *the solution delivers the technical method and the evidence — candidates identified and sized, controls configured, no held item deletable, disposition evidence produced. The* ***retention schedule (how long), records status, legal-hold state, and the authorization to delete*** *are records / compliance / privacy / legal determinations, captured alongside the evidence.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DLCCLEAN-Config.ps1

Purpose: Creates or updates a RETENTION/DELETION policy (and its retention rule) on a NAMED, LIMITED scope — a specific SharePoint site / mailbox / OneDrive, or a named adaptive scope — for the pilot, or creates a retention LABEL (ComplianceTag). Safe by default: with -WhatIf it only reports the intended change; it applies nothing unless -Commit is passed. HARD SAFETY RAILS: it REFUSES a tenant-wide ("All") scope, it requires an explicit -StaticScopeLocation or -AdaptiveScopeName, and it NEVER permanently deletes content — a Delete action only sets the policy's retention rule (deletion happens later on Microsoft's timer, gated by holds and, where configured, disposition review). Enforcement/expansion is a deliberate, hold-checked change-control step (Implementation Phase 3-5), not this tool. HOW TO RUN Preview a delete policy on ONE pilot site (retention period comes from the records schedule): powershell -File .\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `-Workload SharePoint -RetentionAction Delete -RetentionDays 2555` -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -WhatIf Actually create it (still limited to the named pilot site): powershell -File .\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `-Workload SharePoint -RetentionAction Delete -RetentionDays 2555` -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -Workload SharePoint -RetentionAction Delete -RetentionDays 2555 -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -WhatIf Prints the New-RetentionCompliancePolicy / New-RetentionComplianceRule calls it WOULD run, on the named site only. .NOTES Purview — Inactive & Legacy Data Cleanup. Writes gated behind -Commit; refuses "All"; never deletes content. No credentials in code. Expected results: -Commit creates a LIMITED-scope pilot policy; validate scope + holds with DLCCLEAN-Validate.ps1 (PASS = correct scope, no held item deletable). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('Exchange','SharePoint','OneDrive','Teams','Groups')] [string]$Workload = 'SharePoint', [ValidateSet('Keep','Delete','KeepAndDelete')][string]$RetentionAction = 'Delete', Retention period is a RECORDS determination (Req 2.5) — supplied, never invented by this tool. [int]$RetentionDays = 2555, [string]$StaticScopeLocation, # e.g. a specific site URL or mailbox UPN — NEVER 'All' [string]$AdaptiveScopeName, # e.g. 'DLC - Stale Sites - Adaptive Scope' (E5 — verify licensing) Optional label-creation mode instead of a policy: [switch]$CreateLabel, [string]$UserPrincipalName, [switch]$Commit)

How to run: Preview a delete policy on ONE pilot site (retention period comes from the records schedule): powershell -File .\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `-Workload SharePoint -RetentionAction Delete -RetentionDays 2555` -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -WhatIf Actually create it (still limited to the named pilot site): powershell -File .\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `-Workload SharePoint -RetentionAction Delete -RetentionDays 2555` -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\DLCCLEAN-Config.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -Workload SharePoint -RetentionAction Delete -RetentionDays 2555 -StaticScopeLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot' -WhatIf Prints the New-RetentionCompliancePolicy / New-RetentionComplianceRule calls it WOULD run, on the named site

only. .NOTES Purview — Inactive & Legacy Data Cleanup. Writes gated behind -Commit; refuses "All"; never deletes content. No credentials in code. Expected results: -Commit creates a LIMITED-scope pilot policy; validate scope + holds with DLCCLEAN-Validate.ps1 (PASS = correct scope, no held item deletable). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('Exchange','SharePoint','OneDrive','Teams','Groups')] [string]$Workload = 'SharePoint', [ValidateSet('Keep','Delete','KeepAndDelete')][string]$RetentionAction = 'Delete', Retention period is a RECORDS determination (Req 2.5) — supplied, never invented by this tool. [int]$RetentionDays = 2555, [string]$StaticScopeLocation, # e.g. a specific site URL or mailbox UPN — NEVER 'All' [string]$AdaptiveScopeName, # e.g. 'DLC - Stale Sites - Adaptive Scope' (E5 — verify licensing) Optional label-creation mode instead of a policy: [switch]$CreateLabel, [string]$UserPrincipalName, [switch]$Commit)

DLCCLEAN-Discovery.ps1

Purpose: Builds the data-lifecycle baseline before any cleanup: every existing RETENTION policy / label / label policy and its scope; ALL HOLDS in force (litigation holds, eDiscovery case holds, and retain-action retention policies) — the safety inventory that must be respected before any deletion; the INACTIVE MAILBOX inventory (Get-Mailbox -InactiveMailboxOnly); and mailbox/site/OneDrive last-activity + storage signals from Microsoft Graph usage reports. Read-only — makes NO changes and deletes nothing. Use it to see what already exists, what is protected by a hold, and where the ROT is, before you design cleanup. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLCCLEAN-Discovery.ps1 -CsvPath .\DLCCLEAN-Posture.csv Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online (Connect-ExchangeOnline). Storage signals additionally require Microsoft Graph (Connect-MgGraph -Scopes 'Reports.Read.All'). .EXAMPLE .\DLCCLEAN-Discovery.ps1 -CsvPath .\DLCCLEAN-Posture.csv Prints retention policies/labels, the full hold inventory, inactive mailboxes, and storage signals; exports the retention inventory CSV. .NOTES Purview — Inactive & Legacy Data Cleanup. Read-only across S&C PowerShell / Exchange Online / Graph. Expected results: a complete hold inventory + sized stale-data candidates. PASS = holds enumerated and ROT identified before any policy is built. > [CmdletBinding()] param([string]$CsvPath, [int]$StaleDays = 365, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\DLCCLEAN-Discovery.ps1 -CsvPath .\DLCCLEAN-Posture.csv Requires Security & Compliance PowerShell (Connect-IPPSSession) and Exchange Online (Connect-ExchangeOnline). Storage signals additionally require Microsoft Graph (Connect-MgGraph -Scopes 'Reports.Read.All'). .EXAMPLE .\DLCCLEAN-Discovery.ps1 -CsvPath .\DLCCLEAN-Posture.csv Prints retention policies/labels, the full hold inventory, inactive mailboxes, and storage signals; exports the retention inventory CSV. .NOTES Purview — Inactive & Legacy Data Cleanup. Read-only across S&C PowerShell / Exchange Online / Graph. Expected results: a complete hold inventory + sized stale-data candidates. PASS = holds enumerated and ROT identified before any policy is built. > [CmdletBinding()] param([string]$CsvPath, [int]$StaleDays = 365, [string]$UserPrincipalName)

DLCCLEAN-Report.ps1

Purpose: Produces the sized ROT / stale-data candidate list and storage picture the records/storage owners review before signing off cleanup. In the default StaleData mode it uses Microsoft Graph usage reports (SharePoint site, OneDrive account, mailbox usage detail) to list locations with no activity beyond -StaleDays and the storage they consume — the candidate list and the savings estimate. In DispositionPreview mode it reports a named policy's distribution/status (Get-RetentionCompliancePolicy

-DistributionDetail) so you can see where it is deployed before it acts. Read-only — queries reports only; deletes nothing. HOW TO RUN Sized stale-data candidates + storage (needs Graph Reports.Read.All): powershell -ExecutionPolicy Bypass -File .\DLCCLEAN-Report.ps1 -StaleDays 365 -CsvPath .\DLCCLEAN-ROT-Candidates.csv Disposition/deployment preview for a pilot policy: powershell -File .\DLCCLEAN-Report.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -Mode DispositionPreview Requires Microsoft Graph (Connect-MgGraph -Scopes 'Reports.Read.All') and/or S&C PowerShell (Connect-IPPSSession). .EXAMPLE .\DLCCLEAN-Report.ps1 -StaleDays 365 -CsvPath .\DLCCLEAN-ROT-Candidates.csv Lists stale sites/OneDrives/mailboxes (last activity > 365d) with storage GB and exports the sized candidate CSV. .NOTES Purview — Inactive & Legacy Data Cleanup. Read-only reporting from Graph usage reports / retention distribution. No changes made. Expected results: a named, sized ROT candidate list (StaleData) or a deployment/disposition picture (DispositionPreview) for review. > [CmdletBinding()] param([ValidateSet('StaleData','DispositionPreview')][string]$Mode = 'StaleData', [int]$StaleDays = 365, [string]$PolicyName, [string]$CsvPath, [string]$UserPrincipalName)

How to run: Sized stale-data candidates + storage (needs Graph Reports.Read.All): powershell -ExecutionPolicy Bypass -File .\DLCCLEAN-Report.ps1 -StaleDays 365 -CsvPath .\DLCCLEAN-ROT-Candidates.csv Disposition/deployment preview for a pilot policy: powershell -File .\DLCCLEAN-Report.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -Mode DispositionPreview Requires Microsoft Graph (Connect-MgGraph -Scopes 'Reports.Read.All') and/or S&C PowerShell (Connect-IPPSSession). .EXAMPLE .\DLCCLEAN-Report.ps1 -StaleDays 365 -CsvPath .\DLCCLEAN-ROT-Candidates.csv Lists stale sites/OneDrives/mailboxes (last activity > 365d) with storage GB and exports the sized candidate CSV. .NOTES Purview — Inactive & Legacy Data Cleanup. Read-only reporting from Graph usage reports / retention distribution. No changes made. Expected results: a named, sized ROT candidate list (StaleData) or a deployment/disposition picture (DispositionPreview) for review. > [CmdletBinding()] param([ValidateSet('StaleData','DispositionPreview')][string]$Mode = 'StaleData', [int]$StaleDays = 365, [string]$PolicyName, [string]$CsvPath, [string]$UserPrincipalName)

DLCCLEAN-Set-Scope.ps1

Purpose: Reports a retention/deletion policy's current locations, scope, enabled state, and health, and — only with -Commit — adjusts the INCLUDED/EXCLUDED static locations or binds/unbinds a named ADAPTIVE SCOPE (e.g. 'DLC - Stale Sites - Adaptive Scope'), so cleanup can be narrowed to a pilot or expanded one step at a time. Safe by default: -WhatIf reports only; writes require -Commit. HARD SAFETY RAILS: it adjusts SCOPE only — it never changes the retain/delete ACTION, never enables/disables the policy, and never deletes content; and it refuses to ADD an "All" (tenant-wide) location. Re-run DLCCLEAN-Validate.ps1 after any scope change to re-prove the hold cross-check. HOW TO RUN Report current scope/health: powershell -File .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -WhatIf Add a pilot site to the SharePoint scope: powershell -File .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `-AddLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot2' -Workload SharePoint -Commit` `Bind an adaptive scope (E5 — verify licensing): powershell -File .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot'` -BindAdaptiveScope 'DLC - Stale Sites - Adaptive Scope' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -WhatIf Prints the policy's locations/scope/enabled state and the Set-RetentionCompliancePolicy call it WOULD run. .NOTES Purview — Inactive & Legacy Data Cleanup.

Scope changes gated behind -Commit; never changes action/enable; never deletes; refuses "All". No credentials in code. Expected results: -Commit adjusts scope only; Enabled/action unchanged. Re-run DLCCLEAN-Validate.ps1 to re-prove no held item is deletable (PASS). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('Exchange','SharePoint','OneDrive','Teams','Groups')][string]$Workload = 'SharePoint', [string]$AddLocation, [string]$RemoveLocation, [string]$BindAdaptiveScope, [string]$UnbindAdaptiveScope, [string]$UserPrincipalName, [switch]$Commit)

How to run: Report current scope/health: powershell -File .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -WhatIf Add a pilot site to the SharePoint scope: powershell -File .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' `-AddLocation 'https://contoso.sharepoint.com/sites/DLC-Pilot2' -Workload SharePoint -Commit` `Bind an adaptive scope (E5 — verify licensing): powershell -File .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot'` -BindAdaptiveScope 'DLC - Stale Sites - Adaptive Scope' -Commit Requires Security & Compliance PowerShell (Connect-IPPSSession). .EXAMPLE .\DLCCLEAN-Set-Scope.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -WhatIf Prints the policy's locations/scope/enabled state and the Set-RetentionCompliancePolicy call it WOULD run. .NOTES Purview — Inactive & Legacy Data Cleanup. Scope changes gated behind -Commit; never changes action/enable; never deletes; refuses "All". No credentials in code. Expected results: -Commit adjusts scope only; Enabled/action unchanged. Re-run DLCCLEAN-Validate.ps1 to re-prove no held item is deletable (PASS). > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [ValidateSet('Exchange','SharePoint','OneDrive','Teams','Groups')][string]$Workload = 'SharePoint', [string]$AddLocation, [string]$RemoveLocation, [string]$BindAdaptiveScope, [string]$UnbindAdaptiveScope, [string]$UserPrincipalName, [switch]$Commit)

DLCCLEAN-Validate.ps1

Purpose: The defensibility gate. For a named retention/deletion policy it confirms, per gate: (1) the policy is scoped to a LIMITED, named location — not tenant-wide "All"; (2) its retention rule action/period match the intended design; and (3) — the safety centrepiece — it CROSS-CHECKS the policy's scope against every HOLD in the tenant (litigation holds, eDiscovery case holds, and retain-action retention policies) and FLAGS any overlap where an in-scope item could be under hold. Read-only — makes no changes. Deletion must not proceed unless this reports PASS on the hold cross-check. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DLCCLEAN-Validate.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -CsvPath .\DLCCLEAN-Validate.csv Requires Security & Compliance PowerShell (Connect-IPPSSession); Exchange Online (Connect-ExchangeOnline) to include litigation-hold overlap. .EXAMPLE .\DLCCLEAN-Validate.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' Prints per-gate PASS/FAIL for scope, action/period, and the hold cross-check. .NOTES Purview — Inactive & Legacy Data Cleanup. Read-only validation. No changes made. Expected results: PASS = limited scope + expected action + ZERO deletable held items. Any FAIL blocks enforcement until resolved. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$CsvPath, [string]$UserPrincipalName)

How to run: powershell -ExecutionPolicy Bypass -File .\DLCCLEAN-Validate.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' -CsvPath .\DLCCLEAN-Validate.csv Requires Security & Compliance PowerShell (Connect-IPPSSession); Exchange Online (Connect-ExchangeOnline) to include litigation-hold overlap. .EXAMPLE .\DLCCLEAN-Validate.ps1 -PolicyName 'DLC - Delete Inactive Content - Pilot' Prints per-gate PASS/FAIL for scope, action/period, and the hold cross-check. .NOTES Purview — Inactive &

Legacy Data Cleanup. Read-only validation. No changes made. Expected results: PASS = limited scope + expected action + ZERO deletable held items. Any FAIL blocks enforcement until resolved. > [CmdletBinding()] param([Parameter(Mandatory)][string]$PolicyName, [string]$CsvPath, [string]$UserPrincipalName)